

# 6교시: Compute와 process - CPU, process, thread, 명령, exit code

## 수업 목표

- CPU, program, process, thread를 구분한다.
- 명령 실행이 process와 exit code로 이어지는 흐름을 설명한다.
- Docker container와 Kubernetes Pod가 나중에 process 실행 단위를 감싸는 개념임을 preview로 연결한다.

## 50분 흐름

| 시간     | 활동                                      |
|--------|-----------------------------------------|
| 0-5분   | CLI 확인 표 확인                             |
| 5-15분  | compute, program, process, thread 개념 설명 |
| 15-30분 | 명령 실행과 exit code 확인                     |
| 30-40분 | process 관찰과 종료 개념 정리                    |
| 40-50분 | container/Pod preview mapping           |

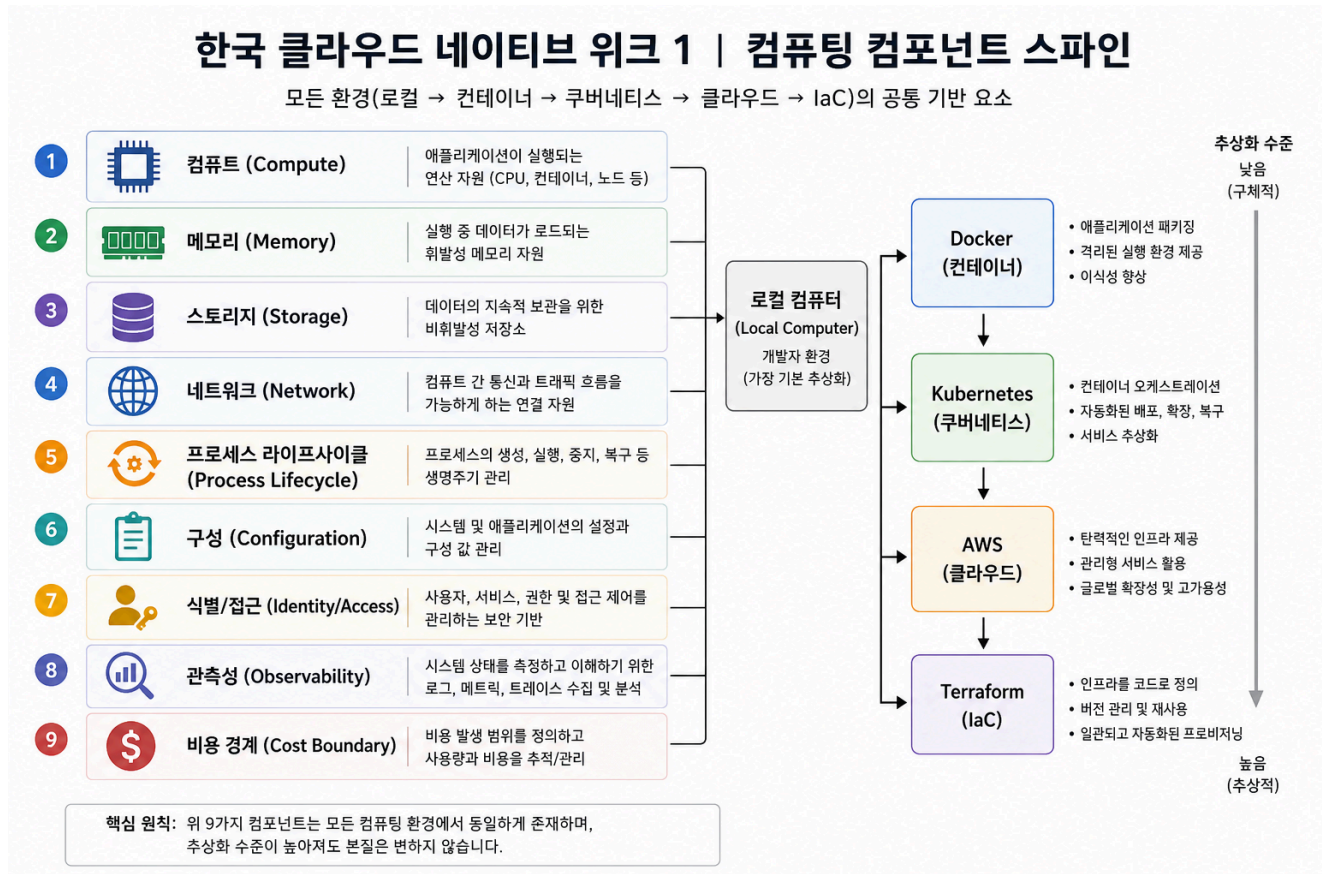
## 0-5분 CLI 확인 표 확인

### 상세 설명

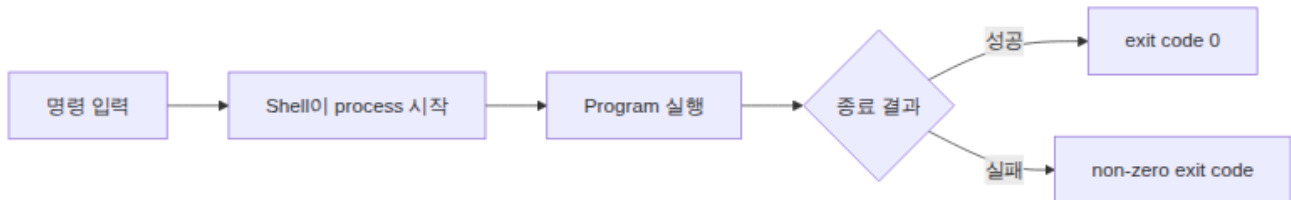
Compute는 계산을 수행하는 자원이다. CPU는 명령을 실행하고, program은 실행 가능한 코드나 스크립트이며, process는 program이 실제로 실행 중인 상태다. thread는 process 내부에서 실행되는 더 작은 흐름이다. Week 1에서는 thread를 깊게 구현하지 않고, "하나의 process 안에도 여러 실행 흐름이 있을 수 있다"는 수준으로 이해한다.

명령을 실행하면 shell은 process를 만들고, process가 끝나면 exit code를 남긴다. exit code 0은 보통 성공, 0이 아닌 값은 실패를 의미한다. 운영 자동화에서 exit code는 매우 중요하다. CI/CD pipeline은 사람처럼 화면을 읽지 않고 exit code로 다음 단계를 계속할지 멈출지 판단한다.

## 시각 자료 1: compute 실행 단위



이 그림은 CPU, program, process가 이후 container와 Pod 개념으로 이어지는 큰 줄기를 보여준다. 오늘날은 container를 실행하지 않지만, "무엇이 실제로 실행 중인가"를 process 기준으로 생각한다.



## 5-15분 compute, program, process, thread 개념 설명

### 핵심 개념

| 개념        | 설명               | Week 1 확인 기록 |
|-----------|------------------|--------------|
| CPU       | 명령을 실행하는 계산 자원   | concept 기록   |
| Program   | 실행 가능한 코드나 명령    | 명령/file 경로   |
| Process   | 실행 중인 program    | ps 출력        |
| Thread    | process 내부 실행 흐름 | concept only |
| Exit code | 명령 종료 결과         | \$?          |

## 시각 자료 2: exit code 관찰 지점

| 캡처할 순간             | 읽어야 할 단서         |
|--------------------|------------------|
| pwd 뒤 echo \$?     | 성공 명령의 exit code |
| ls no-such-file 출력 | 실패 증상 메시지        |
| 실패 뒤 echo \$?      | 자동화가 실패로 판단할 숫자  |

## 15-30분 명령 실행과 exit code 확인

### 시각 자료 3: compute 개념 연결 카드

| 오늘 개념     | 로컬 확인 기록 | 이후 확장          |
|-----------|----------|----------------|
| 명령        | 입력한 명령   | container 명령   |
| process   | ps 출력    | Pod 안 실행 단위    |
| exit code | echo \$? | CI/CD 성공/실패 판단 |

### 명령 절차

```
pwd
echo $?
date
echo $?
ls no-such-file
echo $?
ps
```

### 확인 질문

- ls no-such-file의 증상과 exit code 확인 기록은 무엇인가?
- CI/CD가 exit code를 사용하는 이유는 무엇인가?
- Docker container가 감싸는 핵심 실행 단위는 무엇인가?

## 30-40분 process 관찰과 종료 개념 정리

### 다음 주차 매핑

Docker container는 일반적으로 하나의 주 process를 실행한다. Kubernetes Pod는 container를 감싸고 재시작 정책을 적용한다. AWS ECS, Lambda, EC2도 compute 실행 단위를 제공한다.

## 예상 결과

- pwd와 date 뒤의 echo \$?는 보통 0을 출력한다.
- ls no-such-file은 파일이 없다는 오류를 출력한다.
- 실패한 ls 뒤의 echo \$?는 보통 0이 아닌 값을 출력한다.
- ps는 현재 shell과 실행 중인 process 목록을 보여준다.

## 흔한 오해

| 오해                       | 교정                                           |
|--------------------------|----------------------------------------------|
| 명령과 process는 같은 말이다.     | 명령은 실행 요청이고 process는 실행 중인 상태다.              |
| 화면에 오류가 보이면 항상 프로그램 버그다. | 경로, permission, config, dependency 실패일 수 있다. |
| exit code는 사람이 볼 필요가 없다. | 자동화 시스템은 exit code를 핵심 판단 기준으로 쓴다.           |

## 40-50분 container/Pod preview mapping

### 실습 확인 기록

| 명령              | Observation | Exit code |
|-----------------|-------------|-----------|
| pwd             |             |           |
| date            |             |           |
| ls no-such-file |             |           |
| ps              |             |           |

### 학술 근거와 DevOps 관점

CS systems 교육에서 process와 resource 관리는 운영체제 이해의 핵심이다. DevOps 현업에서는 process가 죽었는지, 재시작되었는지, 어떤 exit code로 끝났는지 확인해야 장애 원인을 좁힐 수 있다. Kubernetes의 restart, readiness, liveness도 결국 process 상태와 연결된다.

### 평가 기준

| 기준     | 2점 확인 기록                                             |
|--------|------------------------------------------------------|
| 50분 참여 | 시간 흐름에 맞춰 설명, 활동, 산출물 작성에 참여했다.                      |
| 증거 산출  | 수업에서 요구한 기록, 명령, 표, 막힘 기록 중 해당 산출물을 구체적으로 남겼다.       |
| 전이 연결  | 오늘 개념이 Week2~5 기술 또는 자기 산출물과 어떻게 연결되는지 한 문장 이상 설명했다. |

### 공식/학술 근거 링크

- MIT Missing Semester, <https://missing.csail.mit.edu/> - shell과 명령-line 작업을 재현 가능한 실무 역량으로 다루는 근거다.

- OSTEP: Operating Systems: Three Easy Pieces, <https://pages.cs.wisc.edu/~remzi/OSTEP/> - process와 filesystem 관찰이 실행 확인 기록이 되는 이유다.
- MDN HTTP Overview, <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview> - browser와 local server 확인을 request/response 확인 기록으로 연결한다.